

The vocabulary

What are these things?

Agentic AI for Economic Research | Lesson 1 | Marco Le Moglie

GitHub repo

The course in four lessons

Lesson 1

Vocabulary

What are these things?

Lesson 2

Second brain

How do agents remember?

Lesson 3

Automations

How do they run without me?

Lesson 4

Control

How do I check their work?

Words for today

What an agent is	What it sees	How you extend it	Where you work
model	prompt	rules file	plan
harness	context window	MCP	API
agent	token	skill	IDE
tool and permission	compaction	sub-agent	CLI

From chat to agent

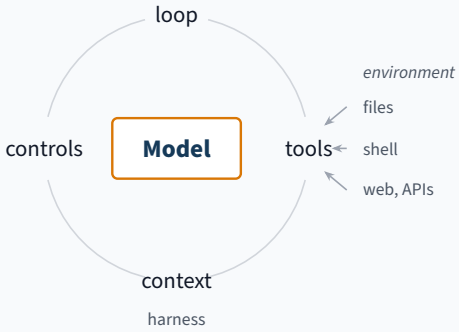
What changes when the model can act

Model plus harness equals agent

Harness

The runtime layer that wraps a language model and turns it into an agent that can act: it runs the loop, exposes tools, manages context and enforces controls.

Macedo (2026); Barbaste et al. (2026)



Same model, different harness: a different agent.

Chatbot vs agent

	Chatbot	Agent
Who runs the loop	You	The harness
Unit of delegation	An answer	A task
Files	Pasted or attached	Read, edited, created
Failure mode	A wrong *answer*	A wrong *action*
Your role	Reader	Supervisor

Different failure mode, different control.

The loop, in a real trace

Observe
reads the do-file

Act
runs Stata.

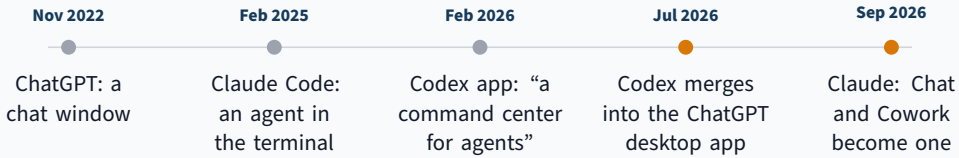
Observe
reads the log. The diff is the typo: `ln(prise)`.

Act
replaces that one line. Minus is the old line, plus is the new one.

Verify
runs it again, and the table is there.

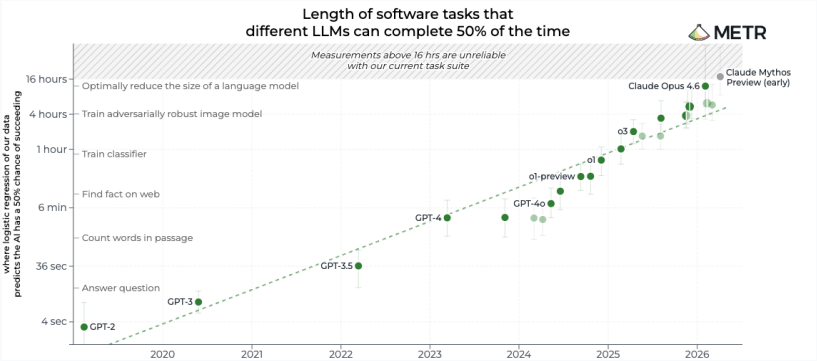
```
> build · deepseek-v4.1-flash-free
→ Read code/analysis.do
$ .../stata-mp -b do code/analysis.do; echo "EXIT:$?"
EXIT:0
→ Read analysis.log
-gen lprice = ln(prise)
+gen lprice = ln(price)
$ .../stata-mp -b do code/analysis.do; echo "EXIT:$?"
EXIT:0
(output written to output/table1.tex)
```

From chat window to command center



Chat and the agent are folding into one harness.

Models finish longer tasks each year



METR, Time Horizons.

What it measures

The length of software tasks, timed on human experts, that a model completes half of the time.

Live: chat vs agent

What we do

The do-file in the demo repository fails.

First, paste it into a chat window and ask for the bug.

Then give the same sentence to an agent opened in the repository folder.

What to look at

Who runs Stata.

Who reads the log.

What you have to check at the end.

Recorded result: what the agent left behind

The change

```
- gen lprice = ln(prise)
+ gen lprice = ln(price)
```

One line in the do-file. The data are untouched, as the rules file asks.

The table runs, but look at the empty base-category rows: “done” still needs a reader. Lesson 4.

The output

	(1) lprice	(2) lprice
l100km	0.0217	0.0193
	(0.0309)	(0.0331)
Weight (lbs.)	0.000377*	0.000406*
	(0.000145)	(0.000153)
Domestic	0	0
	(.)	(.)
Foreign	0.503***	0.486***
	(0.102)	(0.140)
Repair record 1978=1		0
		(.)
Repair record 1978=2		0.0372
		(0.132)
Repair record 1978=3		0.0793
		(0.0923)
Repair record 1978=4		0.102
		(0.114)
Repair record 1978=5		0.146
		(0.141)
Constant	7.095***	6.942***
	(0.199)	(0.215)
Observations	74	69

Standard errors in parentheses

* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

`output/table1.tex` as written by the agent, compiled unedited.

Core vocabulary

What the agent sees, and how you extend it

Prompt vs task

Prompt

"Explain how to cluster standard errors in Stata."

One question, one answer. No state, no side effects.

Task

"Rerun both models with clustered errors, regenerate the table, tell me what changed."

A goal, a workflow, an artefact you can check.

Agents change the unit of delegation.

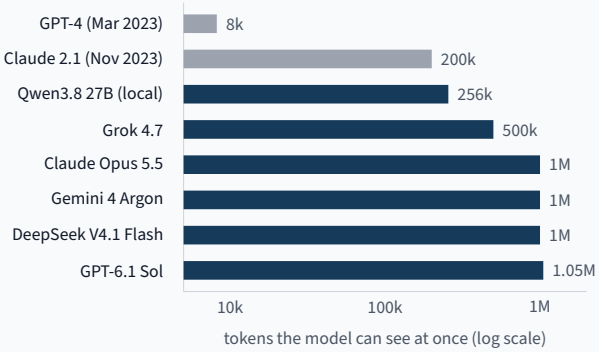
A task an agent can finish names four things

Goal	what should exist at the end
Inputs	which files, which data
Limits	what must not change
Done	how you will check it

Plan, then act

Plan mode writes the steps and waits. Act mode runs the tools. Read the plan before anything changes.

Context window and tokens



Token

The unit models read, write and bill. A 30-page paper is roughly 25k.

Context window

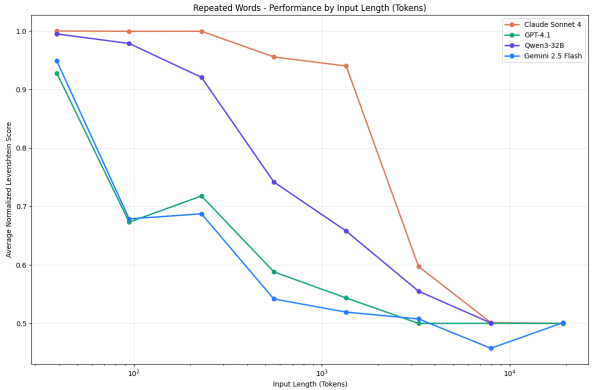
Everything the model sees in one turn: rules, files, tool output, history.

Context rot

Accuracy falls as the window fills. A bigger window is not a better memory.

Past about 200k tokens the model gets worse. Type `/compact`, or `/handoff` and start a new conversation.

Accuracy falls before the window is full



Chroma, July 2025. The test asks the model to copy a list of words. The vertical axis is how close the copy is: 1 means exact, lower means more mistakes. The list gets longer from left to right, and the copy gets worse before the window is full.

Keep the window from filling

Three habits. Each one keeps the model in the next turn from rereading a pile you did not need.

Few skills

Every skill puts its name and description in the window at the start. Write the ones you use. Leave the rest out.

Few MCP servers

Connect a server when the task needs it. Each one adds its tool list before you type.

One message, several jobs

A new question resends the whole window. Ask for the table, the check and the note in the same message.

Compaction

Compaction

Taking a conversation nearing the context window limit, summarizing its contents, and reinitiating a new context window with the summary.

The command to type is `/compact`. Most harnesses also do it on their own.



A summary loses detail. Write the decision in a file. A summary of the chat can drop it, and the next session will not have it.

Handoff, then a new session

Compaction stays in the same chat. Handoff leaves it.

`/handoff`
The agent writes a short file: what was decided, which files matter, what comes next.

New session
You open a fresh conversation and point it at that file. The old window is not sent again.

When to use it
The summary from `/compact` is already lossy, or the chat has wandered off the task.

Rules files: prompts that stay

```
# Instructions for coding agents
[...]
## Layout

- `data/auto.dta`: raw data. Never modify or overwrite.
- `code/analysis.do`: the only analysis script.
- `output/`: generated tables. Safe to overwrite.
[...]
The log is written to `analysis.log` in the repository root.
Read it after every run: Stata reports errors there, not on
the terminal, and the exit code is 0 even when the do-file fails.

## Rules

1. Change the do-file, never the data.
2. Keep variable names short and lower case.
3. Report robust standard errors (`vce(robust)`) unless asked.
4. Before saying a task is done, run the do-file and show the
   last 20 lines of the log.
5. Do not install Stata packages without asking.
```

`AGENTS.md` of the course demo repository. The agent reads it at every session start.

Where the agent is

Layout of the project, what is raw and what is output.

What it must not do

Never touch the data. No new packages without asking.

What "done" means

Run the do-file and show the end of the log.

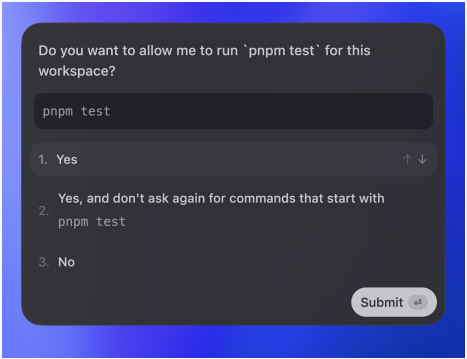
Same idea, different names: `AGENTS.md`, `CLAUDE.md`, `.cursor/rules`.

Tools, permissions, sandbox

- **read** browse and summarise
- **edit** change files in the workspace
- **run** shell commands and scripts
- **network** outside calls, uploads

each step down widens what a mistake can reach

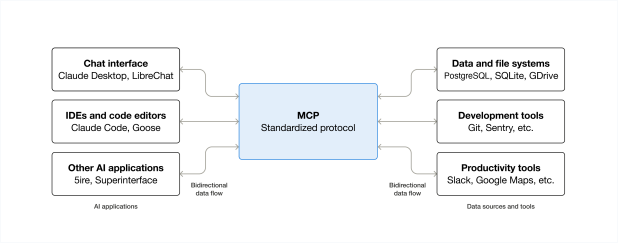
Tool: a function the harness lets the model call.
Permission: the line between proposing and doing.
Sandbox: a place where doing is reversible.



MCP: one plug for many tools

Model Context Protocol

An open standard for connecting AI applications to external systems. The harness is the client; each server exposes tools, data and prompts.



Cost to remember: every connected server adds its tool descriptions to the context before the task starts.

Skills: procedures the agent loads on demand

```
---
name: regression-table
description: Turn stored Stata estimates into a paper-ready LaTeX
  table with esttab. Use when the user asks for a regression table,
  a results table, or to export estimates to LaTeX.
---

# Regression table

1. Check that `esttab` is installed: `which esttab`.
   If missing, stop and ask.
2. Store each model with `estimates store mN` right after
   the regression.
3. Export with:

   esttab m1 m2 using "output/table1.tex", replace ///
       se label booktabs star(* 0.10 ** 0.05 *** 0.01) ///
       stats(N r2, labels("Observations" "R-squared"))

4. Open the `.tex` file and check: one column per model,
   standard errors in parentheses, observations in the last rows.
5. Tell the user which file was written and how many columns
   it has.
```

Skill

A folder with a `SKILL.md` file: a name, a description, instructions, and optional scripts and references.

Agent Skills (2026)

Loaded in three steps

Discovery: name and description only.
Activation: the full file, when a request matches.
Execution: scripts and references, if needed.

Claude Code, Codex and Cursor read a `SKILL.md` file. So do most other harnesses in use.

Sub-agents and hooks

Sub-agent

An agent started by the main agent, with its own context and one narrow job. It hands back a short report and keeps its history.

A reviewer that reads the diff before a commit.

An explorer that searches a large repository.

A Stata runner that returns only the table.

Hook

A fixed command the harness runs at a set moment, whatever the model thinks. It is code; the model cannot skip it.

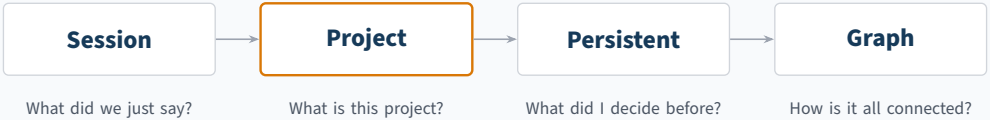
At session start: load the rules and the project state.

After an edit: run a code debugging or a prose check.

Before a commit: block secrets and raw data.

Sub-agents delegate judgement. Hooks remove it.

Memory has layers



The context window is only the first layer.

Live: the skill

What we do

In `demos/lesson1`, type: “Make a regression table.”

What to look at

The agent opens `SKILL.md` before it edits the do-file.

Recorded result: the skill was read when the request matched

```
> build · deepseek-v4.1-flash-free
→ Read code/analysis.do
→ Read .agents/skills/regression-table/SKILL.md
[...]
← Edit code/analysis.do
-esttab m1 m2 using "output/table1.tex", replace se label booktabs
+esttab m1 m2 using "output/table1.tex", replace ///
+    se label booktabs star(* 0.10 ** 0.05 *** 0.01) ///
+    stats(N r2, labels("Observations" "R-squared"))
$ .../stata-mp -b do code/analysis.do; echo "EXIT:$?";
  tail -20 analysis.log
(output written to output/table1.tex)
→ Read output/table1.tex
Table written to `output/table1.tex` with 2 columns (m1, m2),
robust SEs in parentheses, observations (74/69), R-squared,
and `* p<0.10, ** p<0.05, *** p<0.01`.
```

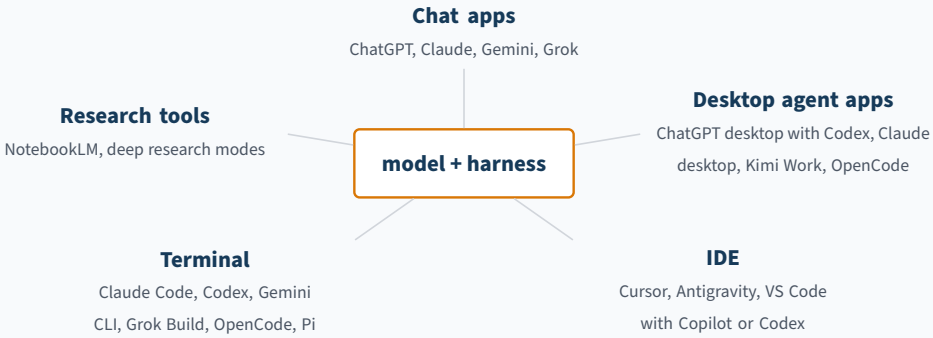
The skill's description says the same thing. The agent read the do-file, then `SKILL.md`, and only then ran Stata.

Stars, statistics and the closing report come from the skill. The prompt never asked for them.

Harnesses in 2026

Where you meet agents, and why the window matters less

Five surfaces, one idea



Pick the surface for the task. The loop underneath is the same.

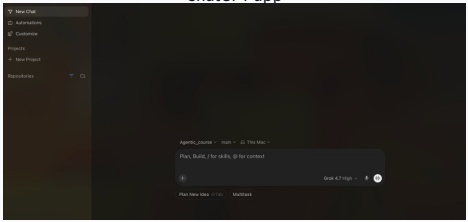
Chat became agent



ChatGPT app



Codex app

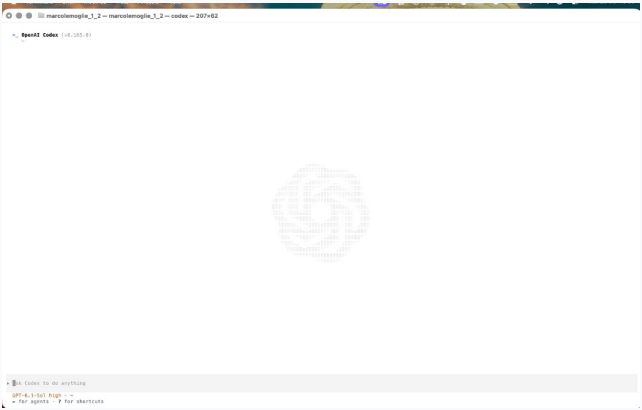


Cursor



OpenCode

The terminal, on its own



Codex in the shell.

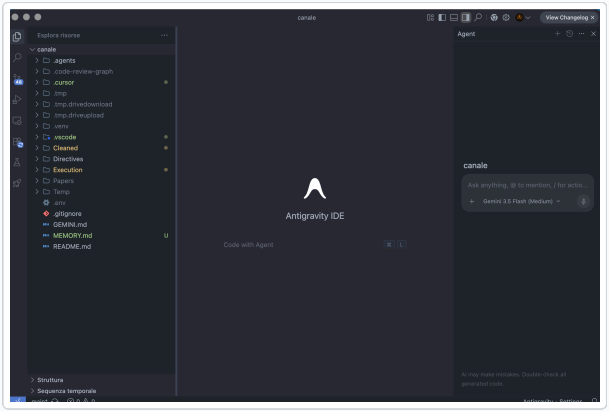
What it is

You type in a shell. The agent reads the folder, runs commands and edits files.

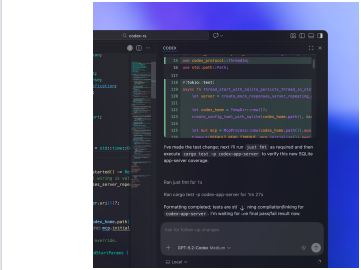
Where it pays

Scriptable. Runs on a server. Every step is text you can log.

IDE agents: review is a diff



Google Antigravity.



Codex inside VS Code.

Every change arrives as a diff you accept or reject, line by line.

Start from the task

If the job is	**start with**
Thinking, writing, reading a paper	**a chat app**
Sources and evidence across many documents	**a research tool**
Work on a project folder, with files you review	**a desktop agent app or an IDE**
Batch jobs, servers, anything you want logged	**a terminal harness**
Data that must not leave the machine	**a local model in an open harness**

Most real work combines two of these.

Models and benchmarks

Who builds them, how to compare them, what people use

Who builds frontier models, October 2026

United States, closed weights

Anthropic	Claude Opus 5.5, Sonnet 5.5, Fable 5.1, Haiku 5.5
OpenAI	GPT-6 Astra, GPT-6.1 Sol, GPT-6 Luna
Google	Gemini 4 Argon
Meta	Muse Wonder, Muse Wonder Full
SpaceXAI	Grok 4.7

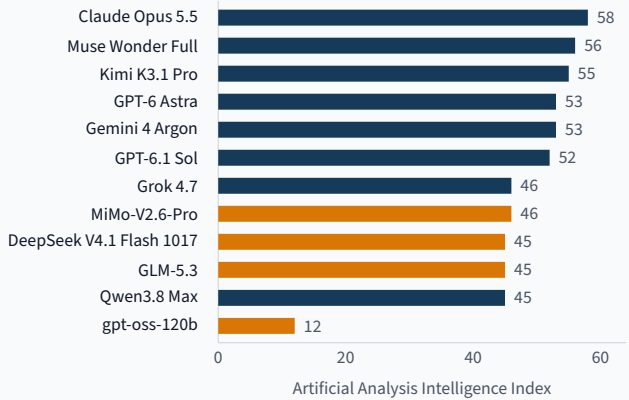
China, mostly open weights

DeepSeek	DeepSeek V4.1 Flash
Z.ai	GLM-5.3, GLM-5.3 Flash
Moonshot	Kimi K3, Kimi K3.1 Pro
Alibaba	Qwen3.8
Xiaomi	MiMo-V2.6 Pro and Flash

Frontier releases now arrive weeks apart: Grok 4.7 on 21 Sep, Opus 5.5 on 22 Sep, Muse Wonder on 12 Oct, Kimi K3.1 Pro on 23 Oct.

Most of these take text and images. A few are text only. The same lab usually ships two: a large model and a cheap fast one, Pro and Flash.

One independent index



Source: Artificial Analysis, Intelligence Index, Oct 2026.

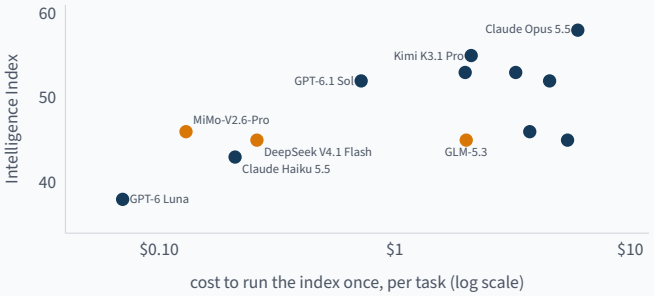
Orange: weights you can download today.
Blue: API only.

The best open-weight model sits 12 points below the top.

Kimi K3.1 Pro is called open-weight at launch; the weights are promised for 4 Nov.

gpt-oss-120b is shown as the strongest US open-weight reference.

Intelligence per dollar



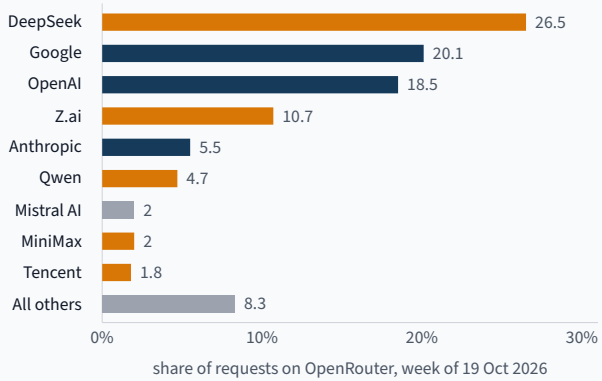
Source: Artificial Analysis, cost per task, Oct 2026.

Same task, cost per task. Opus 5.5: $5.98, score 58. GPT-6.1 Sol: $0.72, score 52. MiMo-V2.6-Pro, open weights: $0.13, score 46.

About 45 times cheaper for twelve points on the Intelligence Index. Whether that trade is good depends on the task.

Do not spend the top model on every step. You pick the tier for the task, or a router in the harness does. Several harnesses ship that as the default.

What people actually use



Source: OpenRouter (2026a).

Orange: Chinese labs. **Blue**: US labs.

Open-weight models have carried more than 62% of OpenRouter tokens every week since July 2026.

Closed weights and open weights

Closed weights

You call the model through an API. The lab keeps the parameters. Fine-tuning them is not available to you.

Open weights

You download the parameters and can fine-tune them for your own task, under the model's licence.

DeepSeek, GLM, Qwen, MiMo, Kimi K3, gpt-oss, Gemma.

Local pays off when

data cannot leave the machine, or the same small task runs thousands of times. Ollama and LM Studio run that smaller model on the laptop.

Local hurts when

The task needs a frontier model, or a long loop.

Live: same harness, another model

What we do

Run the bug-fixing task from the first demo in OpenCode.

Change one flag, `-m`, and run it again with a different model.

What to look at

Time to finish.

Number of steps.

Whether “done” is really done.

Recorded result: one task, three models

Model	Weights	Time	Tool calls	Result
DeepSeek V4.1 Flash (free tier)	open	22 s	9	fixed, table written
GLM-5.3	open	29 s	6	fixed, table written
GPT-6 Luna	closed	42 s	12	fixed, table written

Same harness, same prompt, same repository, same day. Only the `-m` flag changed. A one-line typo is easy for all three; harder tasks are where models separate.

Source: runs on 26 Oct 2026; full logs in `assets/lesson1/traces/`.

Cost, data, reliability

The minimum vocabulary before Lesson 4

Subscription vs API, and what drives the bill

Subscription

Flat monthly price. Limits are real but soft and change without notice. Fine for chat and desktop apps.

API

Pay per token. Easy to measure, easy to overspend. Needed when a harness runs many steps on its own.

List prices per million tokens, input and output:
GPT-6 Luna: $0.10 and $0.50;
GPT-6.1 Sol: $2 and $10;
Claude Opus: 5.5 $4 and $20;
GPT-6 Astra: $10 and $50.

Four levers of an agent's bill

Context	every step resends files, history and tool
Tool calls	more autonomy, more steps
Retries	failed runs, self-checks, sub-agents
Model tier	a 100-fold range between Luna and Astra

A chat subscription often has, in practice, no cap you will hit. An agent on the API pays for every one of these four. The same token usually costs far less inside a subscription than on the API.

What a cost spike looks like

Workload	Example	Tokens, roughly
Chat question	“Explain heteroskedasticity in two lines.”	0.5k
Long reading	“Summarise this 30-page paper.”	25k
Agent task	“Rerun all tables and check them.”	100k to 200k
Long agent run	“Replicate Section 4 end to end.”	0.5M to 2M
Full replication	Paper, online appendix, every do-file and the logs.	above 2M

Does it train on my data, and what is risky

High risk

microdata, administrative or embargoed data, referee reports, IRB-restricted material

Medium

unpublished drafts, grant ideas, coauthors' notes, interview transcripts

Lower

public documentation, published papers, teaching material, synthetic data

Whether it trains depends on:

Plan (consumer or business);
Workspace settings;
API retention policy;
Every tool the agent calls on the way.

Settings change. Check them; do not assume them.

When in doubt: anonymise, use a synthetic copy, or run locally.

Reliability words you will keep hearing

Hallucination	A confident output with no basis: a citation, a number, a variable
Context rot	Quality falls as the context window fills
Reward hacking	The agent meets the check and misses the goal
Sycophancy	The model agrees with you when you are wrong
Automation bias	We trust automated output more than it deserves (Skitka et al., 1999)
Drift	The same prompt gives different results after a model update

The words, one line each

What an agent is

Model: the network that predicts text.

Harness: the runtime around it: loop, tools, context, controls.

Agent: model plus harness, given a task.

Tool and permission: what it can call, and what it may do without asking.

What it sees

Prompt: one request, one answer.

Context window: everything the model sees in one turn.

Token: the unit it reads, writes and bills.

Compaction: old turns replaced by a summary when the window fills.

How you extend it

Rules file: instructions loaded at every start, such as `AGENTS.md`.

MCP: an open standard that plugs tools and data into a harness.

Skill: a `SKILL.md` procedure loaded when a request matches.

Sub-agent: a helper agent with its own context and one job.

Where you work

Plan: the steps, written before any tool runs.

API: pay per token. A chat subscription usually does not.

IDE: the agent beside the files. Each change is a diff.

CLI: the agent in the terminal. You type, it runs commands.

Live: Cruscotto Italia

Open the OpenCode app. Connect this public MCP. No key.

```
https://cruscotto-italia-mcp.agid.workers.dev/mcp
```

Ask it to pick one comune, download the indicators, write a small dataset, and draw two charts.

Before Lesson 2

Install one harness.

ChatGPT desktop with Codex, Claude Code, Cursor or OpenCode.

Connect one MCP server.

Any one: a file system, your calendar, Stata.

Write a ten-line `AGENTS.md` for a real project.

Layout, what must not change, what “done” means. Bring it to Lesson 2.

References I

Agent Skills. Agent skills specification, 2026. URL https://agentskills.io/specification. Accessed 26 October 2026.

Anthropic. Effective context engineering for AI agents. Anthropic Engineering blog, 2025. URL https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

Anthropic. Claude cowork and chat are one claude, September 2026. URL https://support.claude.com/en/articles/16761823-claude-cowork-and-chat-are-one-claude.

Artificial Analysis. LLM leaderboard: Intelligence index v4.3.2, 2026. URL https://artificialanalysis.ai/leaderboards/models. Accessed 26 October 2026.

Paul Barbaste, Tristan Darrigol, Germain Vu, and Tom Wiltberger. Harness engineering: Anatomy, architecture, and evolution of coding agents. a source-code study of eleven systems. arXiv:2609.00006, 2026. URL https://arxiv.org/abs/2609.00006.

Sanderson Oliveira de Macedo. What makes a harness a harness: Necessary and sufficient conditions for an agent harness. arXiv:2606.10106, 2026. URL https://arxiv.org/abs/2606.10106.

METR. Task-completion time horizons of frontier AI models, 2026. URL https://metr.org/time-horizons/. Time Horizon 1.1; page frozen 8 September 2026.

Model Context Protocol. What is the model context protocol (MCP)?, 2026. URL https://modelcontextprotocol.io/docs/getting-started/intro. Accessed 26 October 2026.

OpenAI. ChatGPT is now a partner for your most ambitious work, July 2026a. URL https://openai.com/index/chatgpt-for-your-most-ambitious-work/.

OpenAI. Unlocking the codex harness: How we built the app server, 2026b. URL https://openai.com/index/unlocking-the-codex-harness/.

References II

OpenRouter. LLM rankings: Market share by model author, 2026a. URL https://openrouter.ai/rankings. Week beginning 19 October 2026; data CC BY 4.0.

OpenRouter. The token state of play. Inferred, issue 01, September 2026b. URL https://openrouter.ai/data/newsletter/the-token-state-of-play.

Linda J. Skitka, Kathleen L. Mosier, and Mark Burdick. Does automation bias decision-making? International Journal of Human-Computer Studies 51(5): 991–1006, 1999.